

Experiment 1: Parallel BFS and DFS using OpenMP

This guide explains how to run the Parallel BFS and DFS OpenMP program on both Windows and Linux/Ubuntu systems. The guide assumes you are completely new to programming and computers.

Problem Statement: Design and implement Parallel Breadth First Search (BFS) and Depth First Search (DFS) using OpenMP.

Correct Program Code

```
#include <omp.h>
#include <iostream>
#include <queue>
#include <chrono>

using namespace std;
using namespace std::chrono;

class Node
{
public:
    int value;
    Node* left;
    Node* right;

    Node(int val)
    {
        value = val;
        left = NULL;
        right = NULL;
    }
};

Node* generateTree()
{
    Node* root = new Node(1);

    root->left = new Node(2);
    root->right = new Node(3);

    root->left->left = new Node(4);
    root->left->right = new Node(5);

    root->right->left = new Node(6);
    root->right->right = new Node(7);

    return root;
}

void bfs(Node* root)
{
    if (root == NULL)
        return;

    queue<Node*> q;
    q.push(root);

    while (!q.empty())
    {
        Node* current = q.front();
        q.pop();

        cout << current->value << " ";

        if (current->left != NULL)
            q.push(current->left);

        if (current->right != NULL)
            q.push(current->right);
    }
}
```

```

    }
}

void dfs(Node* root)
{
    if (root == NULL)
        return;

    #pragma omp critical
    cout << root->value << " ";

    #pragma omp parallel sections
    {
        #pragma omp section
        dfs(root->left);

        #pragma omp section
        dfs(root->right);
    }
}

int main()
{
    Node* root = generateTree();

    auto start_bfs = high_resolution_clock::now();

    cout << "BFS Traversal: ";
    bfs(root);

    auto end_bfs = high_resolution_clock::now();

    auto bfs_time =
        duration_cast<microseconds>(end_bfs - start_bfs);

    cout << "\\nBFS Execution Time: "
        << bfs_time.count()
        << " microseconds\\n";

    auto start_dfs = high_resolution_clock::now();

    cout << "\\nDFS Traversal: ";
    dfs(root);

    auto end_dfs = high_resolution_clock::now();

    auto dfs_time =
        duration_cast<microseconds>(end_dfs - start_dfs);

    cout << "\\nDFS Execution Time: "
        << dfs_time.count()
        << " microseconds\\n";

    return 0;
}

```

How to Run on Windows

1. Download and install Code::Blocks with MinGW compiler.
2. Open Code::Blocks.
3. Go to File → New → Empty File.
4. Paste the program code.
5. Save the file as bfs_dfs.cpp.
6. Go to Settings → Compiler → Other Compiler Options.
7. Add: -fopenmp
8. Click OK.
9. Press F9 or Build and Run.

How to Run on Linux / Ubuntu

Install required software:

```
sudo apt update
sudo apt install g++
sudo apt install libomp-dev
```

```
nano bfs_dfs.cpp
```

Compile and run:

```
g++ -fopenmp bfs_dfs.cpp -o bfs_dfs
./bfs_dfs
```

Expected Output

BFS Traversal: 1 2 3 4 5 6 7

BFS Execution Time: 40 microseconds

DFS Traversal: 1 2 4 5 3 6 7

DFS Execution Time: 150 microseconds

Important Viva Questions

Question	Answer
What is BFS?	Breadth First Search visits nodes level by level.
What is DFS?	Depth First Search visits nodes deeply before backtracking.
Why use OpenMP?	OpenMP enables parallel execution using multiple CPU threads.
Why DFS output changes?	Parallel threads execute simultaneously, so order changes.